

Batata

- **Limite de tempo:** 10 minutos
- **Ambiente:** uma GPU (≈16 GB VRAM), sem internet
- **Tamanho da solução:** `solution.ipynb` ≤ 1 MB
- **Armazenamento:** 5 GB

Tarefa

O seu amigo sugere jogar um jogo de adivinhação. Ele, como juiz, escolhe uma palavra oculta de um vocabulário fixo, e o seu objetivo será encontrar a palavra secreta em, no máximo, 30 jogadas. Em cada jogada, o juiz compara duas palavras e indica qual está semanticamente mais próxima da palavra oculta. Todos os jogos começam com o seguinte par fixo `lamp` vs `potato`, porque estas são duas das coisas preferidas do seu amigo. Em seguida, o seu programa propõe uma nova palavra. A palavra vencedora da comparação é mantida e comparada com a sua palavra seguinte. Ganha um jogo no momento em que propõe exatamente a palavra oculta. A correspondência não distingue maiúsculas de minúsculas. Todas as palavras que propuser têm de pertencer a `dataset/vocabulary.json`.

Existe um exemplo completo em `solution.ipynb`, com o protocolo e o carregamento dos dados. Pode alterar a classe `PublicEmbeddingPlayer`. O seu programa é inicializado uma vez e joga todos os jogos numa única execução; o protocolo cria um novo `PublicEmbeddingPlayer` no início de cada jogo.

O Juiz

O seu programa envia um objeto JSON ao Juiz, e o Juiz responde com um objeto JSON.

Um exemplo detalhado, no qual a palavra oculta é mostrada apenas para explicar o protocolo:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}          <- matches: game won
```

As jogadas são numeradas de 1 a 30.

As opções de `verdict` são `first`, o que significa que `word1` está mais próxima, `second`, o que significa que `word2` está mais próxima, ou `same`, o que significa que ambas as palavras estão à mesma distância da palavra oculta.

`winner_word` é a palavra mantida para a comparação seguinte. Perante um veredicto `same`, a primeira palavra permanece.

Dataset

Comum a todas as partições:

- `dataset/vocabulary.json` — 1602 palavras únicas em minúsculas. A palavra oculta é sempre uma destas.
- `dataset/public_embeddings.npy` — `float32`, de dimensão (1602, 2560). Onde a linha `i` corresponde à palavra `i` no vocabulário. Estes são embeddings *públicos*; o juiz utiliza uma representação privada diferente.

As partições são conjuntos de palavras ocultas:

Partição	Palavras	Respostas	Use-a para
<code>test_public</code>	120	<code>dataset/test_public.json</code>	executar a sua solução e autoavaliá-la
<code>test_leaderboard_a</code>	120	ocultas	classificação na leaderboard pública
<code>test_leaderboard_b</code>	120	ocultas	classificação final

Não existe nenhuma partição `train` — uma vez que nada é ajustado a partir de linhas rotuladas.

Modelos fornecidos

São fornecidos com a tarefa dois modelos de embeddings pré-treinados, que podem ser utilizados:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Ambos têm de ser carregados a partir do respetivo caminho local; um identificador do Hugging Face Hub, como "BAAI/bge-m3", desencadeia um download e falha, porque a avaliação é feita offline. Cada diretório contém um `example.py` executável que mostra como fazer uma chamada offline.

As seguintes bibliotecas estão disponíveis: `numpy`, `torch`, `sentence-transformers`. Sem internet, sem downloads e sem outros pacotes.

Saída

Nenhuma. Esta é uma tarefa interativa: a sua solução não escreve nenhum ficheiro de resposta; apenas comunica com o juiz através de `stdin/stdout`, conforme descrito acima.

Pontuação

Um jogo resolvido na jogada t obtém $1.0 - 0.02 \times \max(0, t - 10)$; um jogo não resolvido no prazo de 30 jogadas obtém 0. Assim, as jogadas 1–10 obtêm 1.00, a jogada 20 obtém 0.80 e a jogada 30 obtém 0.60.

A pontuação da sua tarefa é a pontuação média dos jogos $\times 100$, entre 0.00 e 100.00.

O limite de 10 minutos é um orçamento único que abrange o arranque, a preparação e todos os 120 jogos do conjunto de teste.

Como submeter

1. Abra `solution.ipynb`, edite `PublicEmbeddingPlayer` e execute todas as células para se certificar de que funciona.
2. Opcionalmente, pode verificar localmente com: `python local_test.py solution.ipynb --limit 5`. O juiz local utiliza os embeddings *públicos*, pelo que a respetiva pontuação é apenas indicativa.
3. Guarde `solution.ipynb`.
4. Abra o separador Git na barra lateral esquerda do JupyterLab.
5. Coloque `solution.ipynb` em staging (o símbolo + junto ao ficheiro).
6. Introduza uma mensagem de commit e clique em Commit.
7. Clique no ícone da nuvem com uma seta para cima para fazer push.
8. Regresse a esta página Contest e clique em Submit, utilizando uma mensagem de commit que corresponda à que forneceu.

Submeta exatamente um ficheiro, denominado `solution.ipynb`, que abranja toda a preparação e inferência necessária.